

Team RAP twin

RAP twin

Digital Twin Fabric Simulator for Distributed Systems

Software Requirements Specification

Version 1.0

Team Guide: Prof. K. K. Deshmukh (Internal Guide)

Members: Riaan Masoud Attar (Seat No. T231311002)

College Name: K. K. Wagh Institute of Engineering Education and Research, Nashik (An Autonomous Institute affiliated to Savitribai Phule Pune University)

Department: Artificial Intelligence and Data Science

State: Maharashtra

Academic Year: 2024-2025

Revision History

Date	Version	Description	Author
06 October 2026	1.0	Initial Software Requirements Specification. Implemented, in-progress and planned capabilities are marked separately (see Section 3.2).	Team RAP twin

Table of Contents

1.0 Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to describe, completely and unambiguously, the functional and non-functional requirements of RAP twin, a high-fidelity digital twin and planning sandbox for distributed computing fabrics. It is intended for the project guide, the development team, examiners, and any future maintainer, and serves as the agreed reference for what the system must do.

RAP twin lets a researcher or platform engineer describe a distributed fabric (edge, multi-cloud, or hybrid) as static YAML descriptors, merge it with live telemetry, submit multi-stage job DAGs against a choice of scheduling policies, inject deterministic chaos scenarios, and statistically validate policy resilience — all before any of it touches physical infrastructure. The aim is to let intelligent scheduling policies be evaluated safely, repeatedly, and offline, instead of being tested directly against production hardware.

This document describes the system as it stands on the date of issue. Because the project continues to evolve, every capability in this document is labelled Implemented, In Progress or Planned (see Section 3.2).

1.2 Scope

The scope of RAP twin includes:

- Unifying static hardware descriptors (YAML) with streaming live telemetry into a single, continuously synchronized digital twin state (dt.state.DTState), with hot-reloading of on-disk descriptor changes.
- A pluggable multi-strategy planning engine (Greedy, Cheapest-Energy, Resilient/Federated, and RL-Markov/MDP) with a dry-run evaluation mode that predicts Job Completion Time (JCT), energy draw, and SLA risk without committing real resource locks.
- A predictive analytics engine that forecasts per-node utilization, thermal derating risk, and wireless link loss variability.
- A deterministic Chaos Engineering module that injects physical-layer disruptions — link outages, packet noise, thermal throttling, power surges — against the simulated topology.
- Automated background guardrails: a SelfHealingController that reroutes jobs away from degraded nodes, and a ResourceGuardian that reclaims expired reservations.
- A Monte-Carlo benchmarking orchestrator that runs large trial sweeps to statistically validate policy resilience, a what-if capacity analyzer, and a blast-radius analyzer.
- Exporters that serialize twin state into Azure Digital Twins Definition Language (DTDL) and Kubernetes Custom Resource Definitions (CRDs).
- A Flask-based REST API and a React/TypeScript web dashboard for submitting jobs, observing telemetry, and visualizing topology, health, and job history in real time.

Out of scope (current version): real execution of production workloads, authentication/authorization on the REST API, horizontal scaling of the simulator itself across multiple machines, and physical-hardware control. Event-driven ingestion (Kafka/MQTT), containerized GPU resource limits, and live integration with a physical 5G/6G testbed are planned future work (Section 3.2.8).

1.3 Definitions, Acronyms, and Abbreviations

- SRS (Software Requirements Specification): A document that describes what a software system must do.
- DT (Digital Twin): A virtual representation of a physical system (nodes, links, workloads) kept in sync with live telemetry.
- DAG (Directed Acyclic Graph): The multi-stage job representation used to describe workload dependencies and resource requirements.
- JCT (Job Completion Time): Wall-clock time from job submission to completion of its final stage.
- SLA (Service Level Agreement): A completion-deadline and quality-of-service contract a job or node must satisfy.
- MDP (Markov Decision Process): The formal reinforcement-learning model underlying the RL-Markov planning policy.

- MADRL (Multi-Agent Deep Reinforcement Learning): Reinforcement learning across multiple cooperating agents.
- Chaos Engineering: The practice of injecting controlled faults into a system to test its resilience before those faults occur in production.
- Δf (Estimation Deviation): The discrepancy between digital-twin-estimated node compute capacity and actual physical capacity.
- REST (Representational State Transfer): The HTTP/JSON architectural style used by the RAP twin API.
- YAML: Human-readable format used for static node, topology, and job-schema descriptors.
- DTDL (Digital Twins Definition Language): Microsoft Azure's open modeling language for digital twins, used as an interoperability export target.
- CRD (Custom Resource Definition): A Kubernetes extension mechanism used as a second interoperability export target.
- VRAM: Video RAM — one of the resource dimensions (alongside CPU and RAM) tracked per node.
- Blast radius: The set of jobs/nodes affected if a given node or link fails.
- CloudEvents: A vendor-neutral specification for describing event data, used for RAP twin's internal event bus.

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Qi, Q. et al., "A review of digital twins in smart industries," IEEE Access, 2019.
- Fuller, A. et al., "Digital Twin: Enabling Technologies, Challenges and Open Research," IEEE Access, 2020.
- Xu, C. et al., "Digital Twin-Driven Collaborative Scheduling for Heterogeneous Task and Edge-End Resource via Multi-Agent Deep Reinforcement Learning," IEEE JSAC, 2023.
- Dai, Y. et al., "Federated Deep Reinforcement Learning for Task Offloading in Digital Twin Edge Networks," IEEE TNSE, 2024.
- Tran-Dang, H. and Kim, D.-S., "Digital Twin-empowered intelligent computation offloading for edge computing in the era of 5G and beyond," ICT Express, 2025.
- Esen, E. et al., "Chaos experiments in microservice architectures: A systematic literature review," Computer Standards & Interfaces, 2026.
- Poltronieri, F. et al., "ChaosTwin: A Chaos Engineering and Digital Twin Approach for the Design of Resilient IT Services," IEEE CNSM, 2021.
- Flask documentation: <https://flask.palletsprojects.com>; React documentation: <https://react.dev>; Vite documentation: <https://vite.dev>
- R. M. Attar, "RAP twin: A Digital Twin Simulation Platform for Distributed Systems," Project Based Seminar Report, K. K. Wagh Institute of Engineering Education and Research, 2024-2025.
- RAP twin project repository documents: README, documentation/, and paper/main.tex (RAP twin design paper).

1.5 Technologies to be used

- Python 3.12: Core simulation engine, planners, chaos engine, predictive analytics.
- Flask (>=3.0): REST API server (dt/api.py, ui/dashboard.py).
- PyYAML: Parsing of node, topology, and job-schema descriptors.
- jsonschema: Validation of job and node payloads against schemas/*.yaml.
- NumPy / Pandas: Monte-Carlo trial aggregation and statistical reporting.
- Matplotlib: Offline benchmark plotting (tools/policy_benchmark.py).
- rich: CLI formatting for planner/run_plan.py and planner/submit_demo.py.
- pytest, black, isort, ruff: Automated testing and code-quality tooling.
- React 19 + TypeScript + Vite 8: Web dashboard single-page application (web/).

- TailwindCSS 4: Dashboard styling.
- D3.js: Topology and job-history visualizations in the dashboard.
- TanStack Query: Client-side data fetching/caching for the dashboard.
- Docker SDK for Python (optional): Containerized fabric launch via fabric_docker/launch_fabric.py.

1.6 Overview

The SRS contains two main sections, as follows:

- Overall Description: describes the major components of the system, their interconnections and external interfaces (Section 2).
- Specific Requirements: describes the functions of the actors, their roles in the system, the detailed functional and non-functional requirements and the constraints on the system (Section 3).

2.0 Overall Description

2.1 Product Perspective

RAP twin is a new, self-contained local simulation sandbox. It is not a component of a larger system; it sits on top of standard open-source Python/web tooling and adds a closed-loop digital-twin layer in front of conventional scheduling research. A conventional scheduler evaluation requires risking real infrastructure or hand-rolling a simulator; RAP twin removes that risk by giving researchers a sandbox that mirrors a physical fabric's state (via static YAML descriptors and optional live /observe telemetry) without ever issuing commands to real hardware.

The system is organised in layers: a React frontend; a Flask API layer that exposes the REST endpoints; a state layer (DTState, hot-reloading, telemetry merge); a predictive-analytics layer; a planning layer (pluggable strategies); a chaos-and-guardrail layer; and a benchmarking/export/reporting layer. Static descriptors, live overrides, trained RL state, and run reports are stored on the local file system — there is no external database server.

The end-to-end flow is: load/hot-reload topology and node descriptors, merge live telemetry via /observe, submit a job DAG, resolve a placement through the chosen planner (optionally dry-run), optionally inject chaos and let the self-healing controller reroute, sweep the policy across Monte-Carlo trials, and export the resulting twin state to an external control plane.

2.2 Software Interface

- Front End Client: React 19 + TypeScript single-page dashboard (web/), built with Vite, running in a modern web browser (Chrome, Edge, Firefox). It communicates with the backend using HTTP/JSON (fetch) and Server-Sent Events.
- Web Server: Flask applications dt/api.py (core digital twin engine) and ui/dashboard.py (dashboard-facing proxy and convenience endpoints), exposing JSON over REST.
- Data Base Server: None in the current version. Data is kept in file storage: static descriptors (nodes/*.yaml, sim/topology.yaml), a JSON telemetry-override file (sim/overrides.json), and in-memory DTState. A relational store is not currently planned, as the file-based model has proven sufficient at this scale.
- Back End: Python 3.12 with Flask, PyYAML, jsonschema, NumPy, Pandas; planner policies, the chaos engine, and self-healing controllers are pure-Python modules.
- Operating system: Platform independent (developed on Linux; runs on Windows/macOS with Python 3.12 and Node.js).

2.3 Hardware Interface

2.3.1 Client Side

Any computer, tablet, or phone with a modern web browser and network access to the server. No special hardware is required to view the dashboard.

2.3.2 Server Side

A standard x86_64 workstation or server running Python 3.12, with at least 4 CPU cores and 4 GB RAM recommended for comfortable Monte-Carlo sweeps. Development and benchmarking were carried out on an Intel i7-13700K CPU with an NVIDIA RTX 4090 GPU (24GB VRAM); the GPU is tracked only as a cost/resource metadata field for planning and is not required by the core simulation, chaos, or dashboard functionality, which is CPU-bound.

2.4 Product Functions

The main functions of the system are listed below; detailed requirements are in Section 3.2.

- State management: ingest and hot-reload static descriptors, merge streaming telemetry overrides, track per-node/link estimation deviation.
- Predictive analytics: forecast per-node utilization, thermal derating risk, and link-loss variability.

- Planning: route DAG jobs through pluggable scheduling policies (Greedy, Cheapest-Energy, Resilient/Federated, RL-Markov), with a dry-run mode and batch planning.
- Chaos engineering: inject deterministic, time-compressed fault scenarios against the simulated topology; start/stop chaos via REST.
- Self-healing: detect degraded nodes and automatically reroute active job reservations; reclaim expired reservations.
- Benchmarking and analysis: run Monte-Carlo trial sweeps, what-if capacity analysis, and blast-radius analysis.
- Interoperability: export twin state to Azure DTDL and Kubernetes CRDs.
- Visualization: real-time web dashboard showing topology, node health, and job history.

2.5 User Characteristics

User	Characteristics	Typical use
Fabric Operator / Researcher (primary)	Understands distributed-systems concepts (scheduling, SLAs, fault tolerance); comfortable with YAML and REST APIs.	Submits jobs, configures chaos scenarios, reviews the dashboard, runs Monte-Carlo sweeps.
SRE / Platform Engineer	Operates real fabrics; wants to validate a policy change before deploying it.	Uses dry-run planning and what-if analysis to de-risk a scheduling-policy change.
Student / Research user	Basic ML/distributed-systems knowledge; wants to compare scheduling policies.	Benchmarks RL-Markov against heuristic planners using tools/policy_benchmark.py.
Developer / Maintainer	Knows Python, REST APIs, and the pluggable planner interface.	Adds new planner strategies, extends the chaos engine, runs the pytest suite.

2.6 Constraints

- RAP twin itself runs as a single local process/workstation; it simulates a distributed fabric but is not itself horizontally distributed.
- Python's GIL limits raw concurrency of the core simulation loop; background controllers run as threads, not separate processes.
- The file-watcher poll interval (~0.5s) introduces a bounded latency between an on-disk descriptor change and the state engine picking it up.
- The REST API ships without built-in authentication/authorization; it is intended for trusted local or research-network use, not public exposure.
- The Flask development server is not configured for high-availability, multi-instance horizontal scaling out of the box.
- Supported job topology is a DAG of stages with CPU/RAM/VRAM resource requirements; arbitrary non-DAG workload graphs are not supported.

2.7 Architecture Design

RAP twin follows a closed-loop, layered architecture. Static descriptors and streaming telemetry are unified by the Digital Twin Core State Engine (dt.state.DTState), which exposes a state-query interface to the Pluggable Multi-Strategy Planner. The planner routes workload DAGs through Baseline Heuristics, the Resilient/Federated planner, or the RL-Markov planner, optionally in dry-run mode. The Predictive Analytics Engine supplies computed risk metrics (thermal derates, reliability, link loss) to the planner. The Chaos Engine and the Self-Healing/Resource Guardian controllers inject faults into, and heal, the state engine directly, closing the loop. The Interactive Dashboard, REST API, and DTDL/CRD Exporters expose and consume twin state for external operators and control planes.

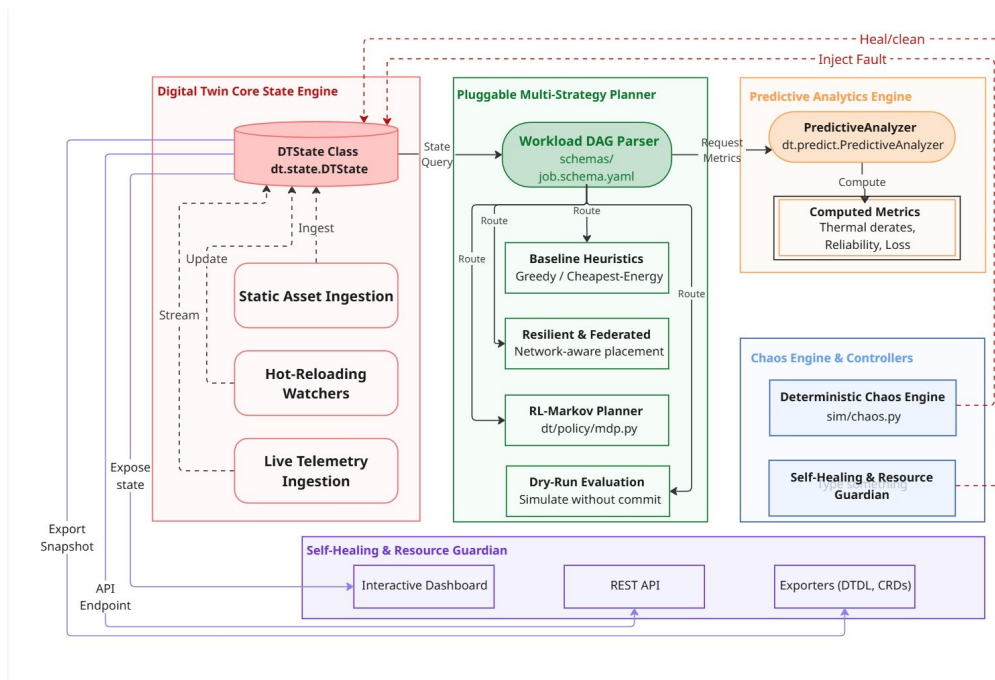


Figure 2.7: Layered closed-loop architecture of RAP twin.

The main components and their responsibilities are:

Layer / Component	Responsibility	Main modules
State layer	Unified state store; merges static descriptors with streaming telemetry; hot-reloads on-disk changes.	dt/state.py, dt/validators.py
Predictive-analytics layer	Computes utilization forecasts, thermal derating, and link-loss risk.	dt/predict.py
Planning layer	Pluggable scheduling strategies and dry-run evaluation; RL/bandit components.	dt/planners.py, dt/policy/*.py
Economics layer	Converts a plan into cost (INR) and carbon (CO ₂) estimates.	dt/cost_model.py, dt/economics.py
Chaos & guardrail layer	Deterministic fault injection; self-healing reroute; reservation reclamation.	sim/chaos.py, dt/chaos_runner.py, dt/self_heal.py
Analysis layer	Monte-Carlo sweeps, what-if capacity analysis, blast-radius analysis.	sim/montecarlo.py, dt/whatif.py
Interoperability layer	Serializes twin state to Azure DTDL and Kubernetes CRDs; CloudEvents bus.	dt/exporters.py, dt/events.py
Presentation layer	REST API and real-time web dashboard.	dt/api.py, ui/dashboard.py, web/src/*

2.8 Use Case Model Description

The use case diagram shows the interactions between the Fabric Operator / Researcher and the system. The Self-Healing Controller, Chaos Engine, and an External Control Plane are shown as supporting actors.

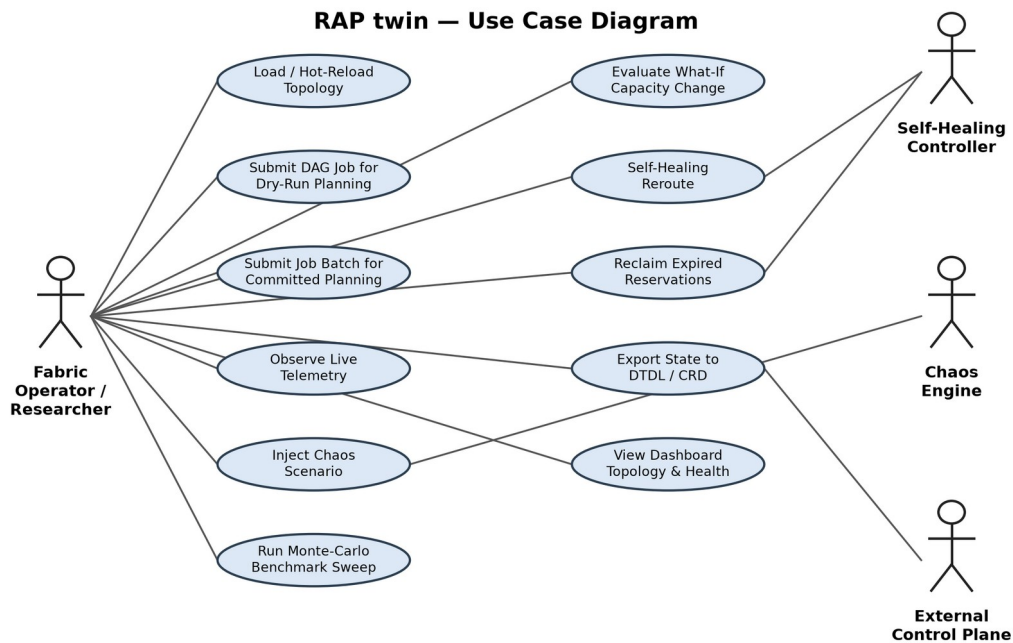


Figure 2.8: Use case diagram.

ID	Use case	Description
UC1	Load / Hot-Reload Topology	Operator edits node/topology YAML on disk; the system detects and ingests the change.
UC2	Submit DAG Job for Dry-Run Planning	User submits a job DAG with dry_run=true and receives predicted JCT/energy/SLA risk without committing a resource lock.
UC3	Submit Job Batch for Committed Planning	User submits multiple jobs for committed placement via /plan_batch.
UC4	Observe Live Telemetry	External or simulated telemetry is POSTed to /observe and merged into DTState.
UC5	Inject Chaos Scenario	Operator starts a deterministic fault scenario (link outage, thermal derate, packet noise) via /chaos/start.
UC6	Self-Healing Reroute	SelfHealingController detects a degraded node and reroutes affected job reservations (system-triggered).
UC7	Reclaim Expired Reservations	ResourceGuardian purges abandoned or timed-out resource reservations (system-triggered).
UC8	Run Monte-Carlo Benchmark Sweep	Operator runs a large trial sweep to statistically evaluate a policy's SLA compliance under chaos.
UC9	Evaluate What-If Capacity Change	Operator forks the twin state, applies a proposed hardware change, and compares outcomes without mutating the live twin.
UC10	Export Twin State to DTDL / CRD	External control plane (Azure Digital Twins / Kubernetes) consumes an exported DTDL or CRD artifact.

2.9 Class and Component Overview

RAP twin is implemented as a set of cooperating Python service modules rather than a classical inheritance-heavy object hierarchy. The key classes and their responsibilities are:

Class	Module	Responsibility
-------	--------	----------------

DTState	dt/state.py	Central unified state store; merges static descriptors with streaming telemetry.
PredictiveAnalyzer	dt/predict.py	Computes utilization forecasts, thermal derating, and link-loss risk.
GreedyPlanner	dt/policy/greedy.py	Baseline first-fit/best-fit placement heuristic.
FederatedPlanner	dt/policy/resilient.py	Network-aware placement with redundant fallback paths.
MarkovPlanner	dt/policy/mdp.py	RL-based planner using an MDP formulation.
RLPolicy	dt/policy/rl_stub.py	ϵ -greedy / TD(0) node-selection policy with persistence.
BanditPolicy	dt/policy/bandit.py	UCB1 multi-armed bandit for execution-format selection.
CostModel	dt/cost_model.py	Computes energy and cost metrics used by the Cheapest-Energy planner.
ChaosEngine / ChaosEvent	sim/chaos.py	Deterministic time-compressed fault injection against the topology.
ChaosRunner	dt/chaos_runner.py	Drives scheduled chaos events against a running DTState instance.
SelfHealingController	dt/self_heal.py	Monitors node health scores and triggers failover rerouting.
ResourceGuardian	dt/self_heal.py	Reclaims expired or abandoned resource reservations.
EventBus	dt/events.py	Publishes CloudEvents-formatted state-change notifications.

2.10 Sequence Diagram

The sequence diagram below shows the main scenario: submitting a job for dry-run planning, then a chaos scenario triggering a self-healing reroute.

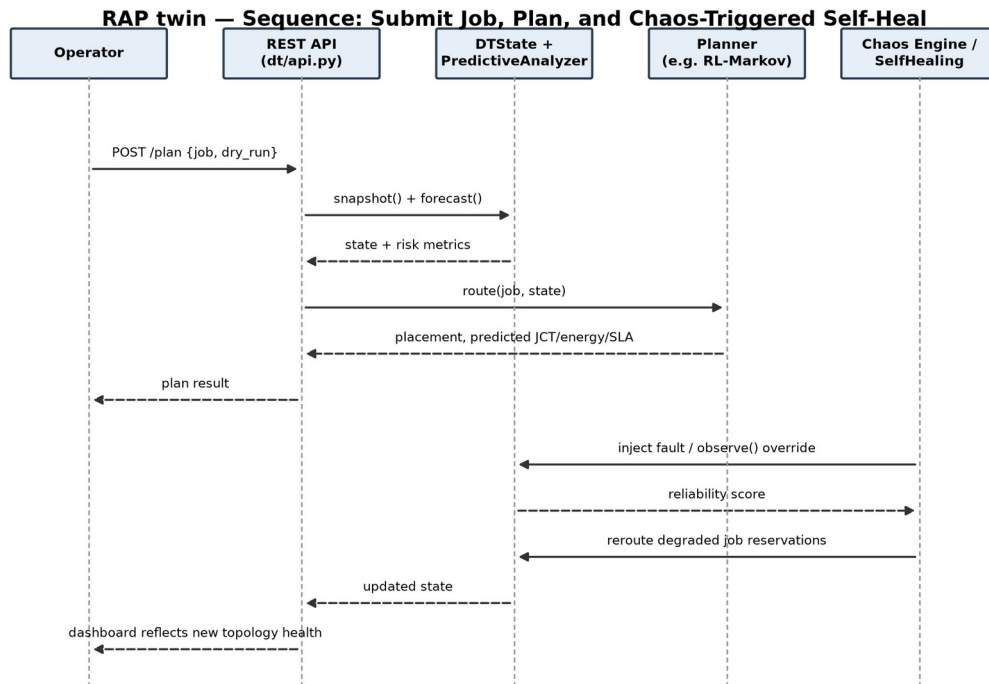


Figure 2.10: Sequence diagram — submit job, plan, and chaos-triggered self-heal.

2.11 Database Design

The current version does not use a database server. Persistent configuration is stored as static YAML files, and runtime state lives in the in-memory DTState object together with a JSON telemetry-override file. The table below is the logical data model of those files — the entities that would become tables if a relational store were added in future work (Section 3.2.8).

2.11.1 Logical Entity Relationships (in place of an ER diagram)

A Topology has many Nodes; a Node has many Links to other Nodes; a Node holds zero or more active Job Reservations; a Job DAG has many Stages, each bound to exactly one Node once planned; a Chaos Scenario has many Chaos Events, each targeting one Node or Link.

2.11.2 Schema

NODE (file in nodes/*.yaml)

Attribute	Type	Description
name (PK)	string	Unique node identifier.
cpu / ram / vram	integer	Hardware capacity along each resource dimension.
arch_flags	list of string	Architecture capability flags used by format-selection policies.
power_draw	float	Baseline power-draw profile used by CostModel.

LINK (part of sim/topology.yaml)

Attribute	Type	Description
key (PK)	string	Composite of the two endpoint node names.
bandwidth	float	Link capacity.

latency / loss_pct	float	Baseline latency and packet-loss percentage.
--------------------	-------	--

JOB (validated against schemas/job.schema.yaml)

Attribute	Type	Description
id (PK)	string	Job identifier.
stages	list of object	DAG stages, each with cpu/ram/vram requirements and allowed formats.
deadline_ms (T_max)	integer	SLA completion deadline.
priority	string	QoS priority used by risk-weighted planners.

OVERRIDE (file sim/overrides.json, via POST /observe)

Attribute	Type	Description
type	enum	node or link — which entity the override targets.
target key	string	Node name or link key being overridden.
changes	object	Telemetry delta merged into DTState (e.g. down, thermal_derate, loss_pct).

MONTE_CARLO_TRIAL (report row in sim/montecarlo.py output)

Attribute	Type	Description
trial_id (PK)	integer	Sequential trial number within the sweep.
strategy	string	Planner strategy evaluated in this trial.
jct / energy / sla_met	float / float / bool	Measured outcome metrics for the trial.

2.12 Assumptions and Dependencies

- Python 3.12+ and Node.js/npm are available on the host running the simulator and dashboard respectively.
- Synthetic workload and channel models (CPU cycles, data sizes, SLA deadlines) are assumed representative of the real fabric behavior being studied.
- Operators are assumed to supply accurate static YAML descriptors; RAP twin does not independently verify physical hardware specifications.
- Core dependencies: Flask, PyYAML, jsonschema, NumPy, Pandas, Matplotlib, rich (Python); React, Vite, TailwindCSS, D3.js, TanStack Query (web).
- Docker is an optional dependency, required only for the containerized fabric launch path (fabric_docker/launch_fabric.py).

3.0 Specific Requirements

3.1 Use Case Reports

UC1: Load / Hot-Reload Topology

Use Case ID	UC1
Use Case Name	Load / Hot-Reload Topology
Actor	Fabric Operator / Researcher
Description	The operator edits node or topology YAML on disk; the system detects the change and ingests it without a restart.
Preconditions	The system is running; nodes/*.yaml and sim/topology.yaml exist.
Main flow	1. Operator edits a node or topology descriptor file on disk. 2. The background file-watcher thread polls descriptor files (~0.5s interval). 3. On detecting a change, DTState re-parses the affected file and merges the updated descriptor into the unified state. 4. Subsequent /snapshot and /plan calls reflect the new topology.
Alternate flows	A1. Malformed YAML: the parser rejects the file and the previous valid state is retained; an error is logged.
Postconditions	The unified state reflects the on-disk topology without requiring a service restart.
Status	Implemented

UC2: Submit DAG Job for Dry-Run Planning

Use Case ID	UC2
Use Case Name	Submit DAG Job for Dry-Run Planning
Actor	Fabric Operator / Researcher
Description	The user submits a job DAG with dry_run=true and receives a predicted placement and risk metrics without committing a resource lock.
Preconditions	A valid topology and node set is loaded; the job DAG conforms to schemas/job.schema.yaml.
Main flow	1. Operator submits a job via POST /plan with dry_run=true and a chosen strategy. 2. The API validates the job against schemas/job.schema.yaml. 3. DTState.snapshot() and PredictiveAnalyzer supply current state and risk metrics. 4. The selected planner computes a placement and predicted JCT, energy, and SLA risk. 5. The result is returned without committing a resource lock and recorded in the recent-plans history.
Alternate flows	A1. Job fails schema validation: HTTP 400 with the validation error. A2. No feasible placement exists under current resource constraints: the planner returns an infeasibility result with the blocking reason.
Postconditions	The operator receives predicted JCT/energy/SLA-risk metrics; no fabric resource state is modified.
Status	Implemented

UC5: Inject Chaos Scenario and Observe Self-Healing

Use Case ID	UC5 / UC6 / UC7
Use Case Name	Inject Chaos Scenario and Observe Self-Healing
Actor	Fabric Operator / Researcher; Chaos Engine, SelfHealingController, ResourceGuardian (supporting)
Description	The operator starts a chaos scenario; the chaos engine injects the fault, and the self-healing controllers detect and recover from it.
Preconditions	The simulated topology and at least one active job reservation exist.
Main flow	1. Operator starts a chaos scenario via POST /chaos/start (e.g., link outage, thermal derate). 2. ChaosEngine writes override payloads to sim/overrides.json and synchronizes via /observe. 3. PredictiveAnalyzer recomputes node reliability and risk scores. 4. SelfHealingController detects the degraded node/link and reroutes affected job reservations to fallback nodes before SLA deadlines expire. 5. ResourceGuardian reclaims any reservations abandoned as a result of the disruption.
Alternate flows	A1. No fallback node is available: the job is flagged as at-risk rather than silently dropped; the dashboard surfaces the warning.
Postconditions	Affected jobs are rerouted (or flagged) before SLA breach; the dashboard reflects the updated topology health and chaos status.
Status	Implemented

UC8: Run Monte-Carlo Benchmark Sweep

Use Case ID	UC8
Use Case Name	Run Monte-Carlo Benchmark Sweep
Actor	Fabric Operator / Researcher; AutoML-style trial orchestrator (supporting)
Description	The operator runs a large automated trial sweep to statistically evaluate a planner's SLA compliance and JCT distribution under chaos.
Preconditions	A topology, job catalogue, and chaos scenario are configured.
Main flow	1. Operator invokes sim/montecarlo.py with a target strategy and trial count (e.g. 1,000 trials). 2. For each trial, the orchestrator generates a synthetic workload profile and runs it under active chaotic conditions. 3. Per-trial JCT, energy, and SLA-hit outcomes are recorded. 4. Results are aggregated into mean/p95/p99 JCT, energy efficiency, and SLA hit-rate statistics and exported to CSV/JSON.
Alternate flows	A1. A trial raises an unexpected error: it is recorded as a failed trial and the sweep continues rather than aborting.
Postconditions	A statistical report exists summarizing policy survival/SLA compliance across the full trial sweep.
Status	Implemented

UC10: Export Twin State to DTDL / CRD

Use Case ID	UC10
Use Case Name	Export Twin State to DTDL / CRD
Actor	Fabric Operator / Researcher; External Control Plane (supporting)
Description	The operator (or an external system) requests a serialized twin state in Azure DTDL or Kubernetes CRD form for interoperability with an external control plane.
Preconditions	A twin state (topology and current node/link status) exists.
Main flow	1. Caller requests GET /standards/dtdl or GET /standards/crds. 2. dt/exporters.py serializes the current DTState into the requested schema. 3. The response is returned as the exported artifact.
Alternate flows	A1. Export requested before any topology is loaded: an empty/minimal artifact is returned rather than an error.
Postconditions	A DTDL or CRD document representing the current twin state is available to the external control plane.
Status	Implemented

3.2 Supplementary Requirements

Status values: Implemented = working in the current code base; In Progress = partly built or being refined; Planned = not yet started.

3.2.1 Functional Requirements

ID	Requirement	Status
FR-01	The system shall parse static node and topology YAML descriptors on startup.	Implemented
FR-02	The system shall hot-reload descriptor files on change via a background poll (~0.5s interval) without requiring a restart.	Implemented
FR-03	The system shall merge streaming telemetry overrides into the unified state via the POST /observe endpoint.	Implemented
FR-04	The system shall validate node, topology, and job payloads against JSON Schema (schemas/*.yaml) before accepting them.	Implemented
FR-05	The system shall track per-node/link estimation deviation (Δf) between twin-predicted and physical capacity.	Implemented
FR-06	The system shall forecast per-node utilization, thermal derating risk, and wireless link-loss variability.	Implemented
FR-07	The system shall route submitted DAG jobs through a pluggable planner interface (dt/policy/*.py).	Implemented
FR-08	The system shall provide a Greedy first-fit/best-fit baseline planner.	Implemented
FR-09	The system shall provide a Cheapest-Energy planner that minimizes power/cost via CostModel.	Implemented
FR-10	The system shall provide a Resilient/Federated planner with redundant fallback-path placement.	Implemented
FR-11	The system shall provide an RL-Markov planner based on an MDP formulation.	Implemented
FR-12	The system shall provide an ϵ -greedy / TD(0) RL policy and a UCB1 bandit policy for	Implemented

	node/format selection.	
FR-13	The system shall support a dry-run evaluation mode that predicts JCT/energy/SLA risk without committing a resource lock.	Implemented
FR-14	The system shall support batch planning of multiple jobs in a single request (POST /plan_batch).	Implemented
FR-15	The system shall inject deterministic, time-compressed chaos scenarios (link outages, packet noise, thermal throttling, power surges).	Implemented
FR-16	The system shall start and stop chaos scenarios via REST (POST /chaos/start, POST /chaos/stop).	Implemented
FR-17	The system shall automatically detect degraded nodes and reroute active job reservations (SelfHealingController).	Implemented
FR-18	The system shall reclaim expired or abandoned resource reservations (ResourceGuardian).	Implemented
FR-19	The system shall run large-scale Monte-Carlo trial sweeps and export aggregated CSV/JSON statistical reports.	Implemented
FR-20	The system shall evaluate a proposed hardware change (what-if analysis) by forking state without mutating the live twin.	Implemented
FR-21	The system shall compute the blast radius of a proposed node/link failure or change.	Implemented
FR-22	The system shall export twin state to Azure Digital Twins Definition Language (DTDL).	Implemented
FR-23	The system shall export twin state to Kubernetes Custom Resource Definitions (CRDs).	Implemented
FR-24	The system shall publish CloudEvents-formatted notifications on state changes via an internal event bus.	Implemented
FR-25	The web dashboard shall render real-time topology, node health, and job history, updated via Server-Sent Events (/stream).	Implemented
FR-26	The system shall support dynamic topology editing (add node, add link) via REST.	Implemented
FR-27	The system shall support launching containerized nodes that mirror nodes/*.yaml via the Docker SDK.	Implemented
FR-28	The containerized fabric launcher shall plumb per-node GPU resource limits into the container runtime.	Implemented
FR-29	The system shall support event-driven telemetry ingestion (Kafka/MQTT) in place of file-watcher polling for large-scale deployments.	Implemented
FR-30	The system shall support live integration with a physical 5G/6G hardware testbed via the /observe endpoint.	Planned
FR-31	The REST API shall support authentication and authorization for non-trusted deployments.	Implemented

3.2.2 External Interface Requirements

User interface. A single-page web dashboard with a topology map, node-health panel, job-submission and dry-run form, chaos-control panel, recent-plans/job-history list, and side-by-side strategy comparison view.

Software interfaces (REST API). All requests and responses use JSON. Representative endpoints:

Endpoint	Method	Purpose
/health	GET	Liveness check.
/snapshot	GET	Current unified twin state.

/observe	POST	Merge a live telemetry override (node or link) into DTState.
/plan	POST	Route a single DAG job through the selected planner (supports dry_run).
/plan_batch	POST	Route multiple DAG jobs in one request.
/chaos/start, /chaos/stop	POST	Start / stop a deterministic chaos scenario.
/whatif	POST	Evaluate a proposed capacity change without mutating the live twin.
/blast_radius	POST	Compute the impact set of a proposed node/link failure.
/standards/dtdl, /standards/crds	GET	Export twin state as Azure DTDL / Kubernetes CRDs.
/stream, /events	GET	Server-Sent Events feed of state-change notifications for the dashboard.
/bundle	GET	Export a consolidated snapshot bundle for offline analysis.

Hardware interfaces. See Section 2.3. Communication interfaces. HTTP/1.1 between browser and server; JSON data format; default backend port matching the Flask app configuration.

3.2.3 Performance Requirements

- Baseline heuristic planners (Greedy, Cheapest-Energy) shall return a placement decision in under 5 ms per job under typical load.
- A Monte-Carlo sweep of 1,000 trials shall complete within a practical local benchmarking session (minutes, not hours) on the reference workstation configuration.
- The file-watcher hot-reload shall pick up an on-disk descriptor change within one poll interval (~0.5s).
- The dashboard shall reflect a chaos-triggered reroute via the SSE stream without a manual page refresh.

3.2.4 Reliability and Accuracy

The following benchmark results (reference fabric: nodes/*.yaml, job catalogue jobs/jobs_10.yaml limited to the first 6 jobs, under the 'core_link_cut' chaos scenario at deadline_scale=0.28) illustrate the resilience gap between baseline heuristics and the resilient/RL-based planners:

Strategy	Mean JCT (s)	p95 JCT (s)	Energy (J)	SLA Hit Rate
Greedy	1.05	1.26	13.0	100.0%
Cheapest-Energy	1.05	1.26	13.0	100.0%
Resilient	1.27	1.72	13.0	83.3%
RL-Markov	1.14	1.37	16.2	100.0%

Measured at commit 2e7962d, seed 7 (103.2s). Mean JCT and Energy measured under normal operation (tools/policy_benchmark.py); p95 JCT and SLA Hit Rate measured under the core_link_cut chaos scenario at deadline_scale=0.28, matching ci/gate.yaml (tools/ci_gate.py). These are two different operating conditions, not one unified run. Reproduce with: python -m tools.benchmark_srs_table --seed 7.

- A single degraded node or link shall not cause an unhandled exception; SelfHealingController shall detect and reroute, or flag the job as at-risk, before its SLA deadline elapses.
- A failure of one Monte-Carlo trial shall not abort the sweep; it shall be recorded as a failed trial and the sweep shall continue.
- The automated test suite (97 pytest test functions across tests/) shall pass before a release.

3.2.5 Security Requirements

- The REST API does not implement authentication or authorization by default and must not be exposed on an untrusted network without an external reverse proxy / auth layer.

- Input payloads (nodes, jobs, observations) are validated against JSON Schema before being merged into state, to reject malformed or unsafe input.
- Node and job names are validated (`valid_name()`) to reject names that could be used for path or injection attacks.
- Container launches (`fabric_docker/launch_fabric.py`) only pull images from configured sources; no arbitrary remote code execution is exposed over the REST API.

3.2.6 Usability Requirements

- The web dashboard shall present topology, node health, and job history without requiring the operator to read raw JSON responses.
- The dry-run mode shall let an operator preview a placement's predicted outcome before committing any resource lock.
- CLI tools (`planner/run_plan.py`, `planner/submit_demo.py`) shall present rich, human-readable output rather than raw JSON for interactive use.
- API errors shall use suitable HTTP status codes (400, 404, 500) with a clear message.

3.2.7 Maintainability and Portability

- New scheduling strategies shall be addable by implementing the pluggable planner interface under `dt/policy/` without modifying the core `DTState` or API layer.
- Code quality is enforced via `black`, `isort`, and `ruff`; functional correctness is covered by the `pytest` suite (97 test functions) under `tests/`.
- The backend is pure Python 3.12 and the frontend is a standard Vite/React build; both run on any major OS (Linux, macOS, Windows) without platform-specific dependencies beyond optional Docker support.

3.2.8 Design Constraints, Open Issues, and Planned Work

Because the project continues to evolve, the following items are known and will be reflected in later versions of this SRS:

- Event-driven telemetry ingestion (Kafka/MQTT) to replace file-watcher polling at scale (planned).
- GPU resource limits in the containerized fabric launcher — currently approximated by CPU/mem limits only (planned; noted as future work in `fabric_docker/launch_fabric.py`).
- Live integration with a physical 5G/6G hardware testbed via the `/observe` endpoint, beyond synthetic workload and channel models (planned).
- Authentication/authorization on the REST API for non-trusted deployments (open issue; currently out of scope — see Section 1.2).
- High-availability / multi-instance horizontal scaling of the simulator itself (open issue; current design is single-workstation).